Pediu Patrão

Dashboard Administrativo

Documentação do projeto

Projeto Pediu Patrão

1. Introdução

O Pediu Patrão é um sistema de pedidos para lanchonete/pizzaria. O sistema registra clientes, produtos, pedidos, descontos e cancelamentos, com controle de acesso por perfil (Administrador, Gerente e Atendente) e auditoria de todas as ações.

O Dashboard Administrativo integrado ao sistema de pedidos. O painel consome dados de uma API REST projetada e implementada para isso, e permite ao responsável pelo estabelecimento acompanhar o desempenho das vendas, ver a situação dos pedidos e identificar tendências da operação, como apoio à tomada de decisões.

2. Objetivo

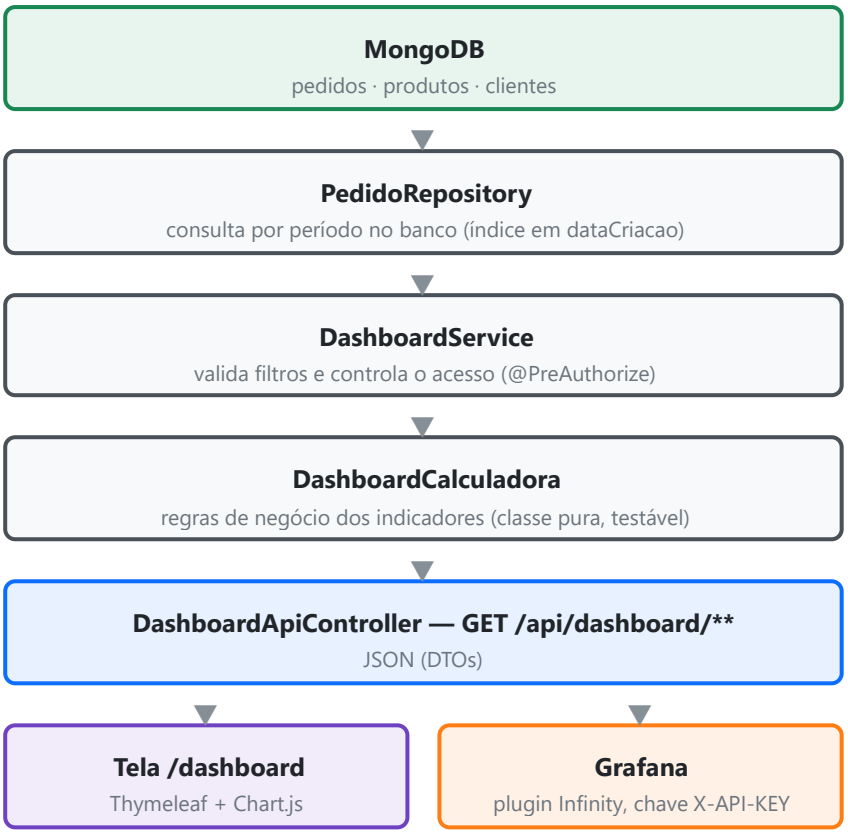
Analisar os requisitos, projetar e implementar o painel administrativo, aplicando conceitos de análise e projeto de software, modelagem de dados, desenvolvimento de APIs e integração entre sistemas.

3. Requisitos do enunciado e onde são atendidos

Requisito	Descrição	Onde está neste documento
RF01	Indicadores gerais de vendas	Parte 1 (seção 2), Parte 2 (`/resumo`), Parte 4 (seção 4)
RF02	Análise de vendas (gráficos)	Parte 2 (`/vendas` , `/vendas-por-hora` , `/vendas-por-categoria`), Parte 4 (seção 5)
RF03	Situação dos pedidos	Parte 2 (`/status` , `/pedidos-recentes`), Parte 4 (seções 5 e 6)
RF04	Produtos mais vendidos	Parte 2 (`/produtos-mais-vendidos`), Parte 4 (seção 6)
RF05	Filtros de consulta	Parte 2 (parâmetros comuns), Parte 4 (seção 7)
RF06	Integração com a API	Parte 2 inteira; a tela e o Grafana consomem a mesma API
RF07	Atualização e tratamento de dados	Parte 2 (erros), Parte 3 (testes de erro), Parte 4 (seções 8 e 9)

4. Arquitetura da solução

A API é a **fonte única dos números**: a tela do sistema e o Grafana apenas desenham o que ela devolve. Se uma regra mudar (por exemplo, o que conta como faturamento), a mudança é feita em um só lugar.



Camada	Componente	Responsabilidade
Dados	`PedidoRepository`	Consulta `{dataCriacao: {$gte, $lt}}` no MongoDB, só com os campos necessários (projeção)
Regra de negócio	`DashboardCalculadora`	Calcula indicadores, séries e ranking; não depende de banco nem de framework
Serviço	`DashboardService`	Valida filtros, busca os dados, monta as respostas e aplica a segurança
Contrato	`DashboardDto`	Formato das respostas JSON (records Java)
API	`DashboardApiController`	8 endpoints REST, somente leitura
Segurança	`SecurityConfig`, `DashboardApiKeyFilter`	Acesso por perfil e chave de API para o Grafana
Apresentação	`dashboard.html`, `dashboard.js`	Tela do dashboard dentro do sistema
Visualização externa	`grafana/`	Docker Compose com o painel já configurado

5. Tecnologias

Área	Tecnologia
Linguagem e framework	Java 21, Spring Boot 3.4 (Web, Data MongoDB, Security, Thymeleaf, Mail)
Banco de dados	MongoDB (Atlas em uso; MongoDB 7 em Docker nos testes)
Interface	Thymeleaf, Bootstrap 5, Chart.js 4
Visualização externa	Grafana 11.3 + plugin Infinity 3.7.1, via Docker Compose

Testes	JUnit 5, Mockito, Spring Security Test (MockMvc), Testcontainers
Ferramentas	Maven (wrapper), Docker Desktop, IntelliJ IDEA

Requisitos funcionais e não funcionais

1. Introdução

1.1 Objetivo

Este documento especifica os requisitos do **Dashboard Administrativo** integrado ao sistema de pedidos Pediu Patrão. Com o painel, o responsável pelo estabelecimento acompanha o desempenho das vendas, a situação dos pedidos e as tendências da operação, para apoiar a tomada de decisões.

1.2 Escopo

Dentro do escopo - API REST somente leitura que calcula e disponibiliza os indicadores (`/api/dashboard/**`). - Tela de dashboard dentro do próprio sistema (`/dashboard`). - Painel equivalente no Grafana, que consome a mesma API. - Gerador de dados de demonstração para testes e apresentação.

Fora do escopo - Cadastro e alteração de pedidos, clientes e produtos. Essas funções já existem no sistema e o dashboard só as **lê**. - Previsão de vendas, metas e exportação para PDF ou Excel (ver seção 7).

1.3 Atores (perfis de acesso)

Ator	Descrição	Acesso ao dashboard
Gerente	Responsável pela operação da loja	Tela e API
Administrador	Gestão de usuários e do sistema	Tela e API
Atendente	Registra pedidos no balcão/telefone	**Sem acesso**
Grafana (sistema externo)	Ferramenta de visualização	Somente API, com chave `X-API-KEY`

1.4 Definições

Termo	Significado
Pedido válido	Pedido cujo status não é `CANCELADO`
Faturamento	Soma da receita líquida (após desconto) dos pedidos válidos
Ticket médio	Faturamento ÷ quantidade de pedidos válidos
Taxa de cancelamento	Pedidos cancelados ÷ total de pedidos do período × 100
Período anterior	Intervalo imediatamente anterior, com o mesmo número de dias do período consultado
KPI	*Key Performance Indicator* — indicador-chave de desempenho

2. Requisitos funcionais

Os sete requisitos do enunciado (RF01 a RF07) foram detalhados em sub-requisitos verificáveis.

RF01 — Indicadores gerais de vendas · *Essencial*

O sistema deve exibir os indicadores consolidados do período filtrado.

ID	Descrição
RF01.1	Exibir o **faturamento** do período
RF01.2	Exibir a quantidade de **pedidos válidos** e o **total de pedidos** (incluindo cancelados)
RF01.3	Exibir o **ticket médio**
RF01.4	Exibir a quantidade de **pedidos cancelados** e a **taxa de cancelamento**
RF01.5	Exibir a quantidade de **itens vendidos** (unidades)
RF01.6	Exibir o **total de descontos concedidos** em reais
RF01.7	Exibir o **tempo médio** entre a entrada do pedido e a saída (entrega ou retirada)
RF01.8	Exibir a **variação percentual** do faturamento e dos pedidos em relação ao período anterior
RF01.9	Exibir o **horário de pico** e o **produto mais vendido** do período

Critérios de aceitação - Com pedidos de R$ 99,00 e R$ 45,00 válidos e um cancelado, o faturamento é R$ 144,00 e o ticket médio é R$ 72,00. - Período sem pedidos mostra zero (sem erro de divisão por zero), e o tempo médio aparece como "Sem dados". - Se o período anterior não tiver vendas, a variação aparece como "sem base no período anterior".

RF02 — Análise de vendas (gráficos) · *Essencial*

ID	Descrição
RF02.1	Exibir a **evolução** do faturamento e da quantidade de pedidos ao longo do tempo
RF02.2	Permitir agrupar a evolução por **dia, semana ou mês**
RF02.3	Exibir a distribuição de pedidos por **hora do dia** (0h a 23h)
RF02.4	Exibir o **faturamento por categoria** de produto (Pizza, Esfiha, Bebida...)

Critérios de aceitação - Dias, semanas ou meses sem venda aparecem com valor zero, para a linha do gráfico não "pular" períodos. - A semana começa na segunda-feira; o mês, no dia 1.

RF03 — Situação dos pedidos · *Essencial*

ID	Descrição
RF03.1	Exibir a quantidade e o percentual de pedidos em **cada status** (Recebido, Em preparação, Pronto, Saiu para entrega, Retirado, Finalizado, Cancelado)
RF03.2	Listar os **últimos pedidos** do período, com data/hora, cliente, itens, status e valor
RF03.3	Oferecer um atalho para a lista completa de pedidos do sistema

Critérios de aceitação - Todos os status aparecem na resposta, mesmo com quantidade zero, na ordem do fluxo do pedido. - Pedidos de cliente não encontrado aparecem como "Cliente não identificado".

RF04 — Produtos mais vendidos · *Essencial*

ID	Descrição
RF04.1	Exibir o **ranking** dos produtos mais vendidos, ordenado por quantidade (desempate pelo faturamento)
RF04.2	Para cada produto, exibir posição, nome, categoria, quantidade e receita
RF04.3	Permitir definir o **tamanho do ranking** (padrão 5, de 1 a 50)

Critérios de aceitação - Itens de pedidos cancelados não entram no ranking.

RF05 — Filtros de consulta · *Essencial*

ID	Descrição
RF05.1	Filtrar por **período** (data inicial e data final)
RF05.2	Filtrar por **status** do pedido
RF05.3	Filtrar por **categoria** de produto
RF05.4	Oferecer **atalhos de período**: hoje, 7 dias, 30 dias, 90 dias e mês atual
RF05.5	Permitir **limpar** os filtros e voltar ao padrão (últimos 30 dias)
RF05.6	Carregar as opções de status e categoria **a partir da API**
RF05.7	Manter os filtros na **URL**, para que atualizar a página ou compartilhar o link preserve a consulta
RF05.8	Aplicar o mesmo filtro a **todos** os indicadores, gráficos e tabelas da tela

Critérios de aceitação - Sem filtro de data, o período é dos últimos 30 dias até hoje. - Data inicial posterior à final é recusada com a mensagem "A data inicial não pode ser posterior à data final." - Período maior que 366 dias é recusado.

RF06 — Integração com a API · *Essencial*

ID	Descrição
RF06.1	Disponibilizar os dados do dashboard por uma **API REST em JSON**
RF06.2	Todos os endpoints aceitam os mesmos filtros (`inicio` , `fim` , `status` , `categoria`)
RF06.3	A tela deve obter **todos** os números pela API (nenhum cálculo de indicador no navegador)
RF06.4	Permitir que **ferramentas externas** (Grafana) consumam a API sem login por formulário

Endpoints (parâmetros, exemplos e erros em API.md)

Método	Endpoint	Atende

GET	`/api/dashboard/resumo`	RF01
GET	`/api/dashboard/vendas?agrupamento=DIA|SEMANA|MES`	RF02.1, RF02.2
GET	`/api/dashboard/vendas-por-hora`	RF02.3
GET	`/api/dashboard/vendas-por-categoria`	RF02.4
GET	`/api/dashboard/status`	RF03.1
GET	`/api/dashboard/pedidos-recentes?limite=10`	RF03.2
GET	`/api/dashboard/produtos-mais-vendidos?limite=5`	RF04

Método	Endpoint	Atende
GET	`/api/dashboard/filtros`	RF05.6

RF07 — Atualização e tratamento de dados · *Essencial*

ID	Descrição
RF07.1	Atualizar os dados **automaticamente** a cada 60 segundos, com opção de desligar
RF07.2	Permitir atualização **manual** (botão "Atualizar") e mostrar o horário da última atualização
RF07.3	Exibir indicador de **carregando** durante a consulta
RF07.4	Exibir **mensagem de erro** compreensível, com opção "Tentar novamente", quando a API falhar
RF07.5	Exibir "Nenhum dado para os filtros selecionados" em gráficos e tabelas vazios
RF07.6	**Validar** os filtros na tela e na API, devolvendo HTTP 400 com a mensagem do erro
RF07.7	Redirecionar para o login quando a sessão expirar (HTTP 401)
RF07.8	Tratar **dados antigos**: pedidos de produto único e status com nomes antigos (`EM_PREPARO` , `ENTREGUE`)
RF07.9	Cancelar uma consulta em andamento quando o usuário aplicar um novo filtro

RF08 — Dados de demonstração · *Desejável*

ID	Descrição
RF08.1	Gerar pedidos fictícios de uma pizzaria de médio porte para um período configurável (padrão 8 meses)
RF08.2	Não duplicar dados se a geração for executada novamente
RF08.3	Remover os dados fictícios sem afetar os dados reais

3.Requisitos não funcionais

3.1 Segurança

ID	Requisito	Prioridade
RNF01	A tela `/dashboard` deve ser acessível somente aos perfis **GERENTE** e **ADMIN**	Essencial
RNF02	A API `/api/dashboard/**` deve aceitar apenas GERENTE, ADMIN e o leitor técnico do Grafana	Essencial
RNF03	A autorização deve ser verificada em **duas camadas**: por rota (Spring Security) e no serviço (`@PreAuthori`	Essencial `ze`)
RNF04	A API do dashboard deve ser **somente leitura**: qualquer método diferente de GET é negado	Essencial
RNF05	A chave de API do Grafana deve: valer só para `GET /api/dashboard/**` ; não criar sessão; ser comparada em tempo constante; e ficar desligada quando não configurada	Essencial
RNF06	Chaves e senhas não devem ficar no código-fonte; devem vir de **variáveis de ambiente**	Importante
RNF07	Respostas de erro não devem expor detalhes técnicos (stack trace, nomes de classes)	Essencial
RNF08	A API deve devolver **DTOs**, nunca a entidade completa do pedido (dados internos, como motivos de cancelamento, não podem vazar)	Importante
RNF09	Textos vindos do banco devem ser escapados antes de ir para o HTML (proteção contra XSS)	Essencial
RNF10	Acessos negados devem ser registrados na auditoria do sistema	Importante

3.2 Desempenho

ID	Requisito	Prioridade
RNF11	Cada endpoint deve responder em até **2 segundos** para um período de 8 meses com cerca de 13 mil pedidos	Importante
RNF12	O filtro de período deve ser feito **no banco**, usando índice no campo `dataCriacao`	Essencial
RNF13	A consulta deve trazer do banco **apenas os campos** usados nos cálculos (projeção)	Importante
RNF14	A tela deve buscar os dados dos endpoints **em paralelo**	Importante
RNF15	A atualização automática deve **pausar** com a aba do navegador oculta	Desejável
RNF16	O período máximo de consulta é **366 dias**, e o tamanho dos rankings e listas é limitado a **50** itens	Importante

> Medição realizada (07/11/2026, MongoDB Atlas): consulta de 8 meses passou de ~4,5 s para ~1,1 s após o índice e a projeção; consulta padrão de 30 dias, ~0,6 s.

3.3 Confiabilidade e exatidão

ID	Requisito	Prioridade
RNF17	Valores monetários devem ser calculados com `BigDecimal` e arredondamento comercial (HALF_UP, 2	Essencial casas)
RNF18	Indicadores com divisão (ticket médio, taxas, variação) não podem falhar com denominador zero	Essencial

RNF19	Os números da tela própria e do Grafana devem ser **idênticos**, pois vêm da mesma API	Essencial

3.4 Usabilidade

ID	Requisito	Prioridade
RNF20	Interface em **português**, com valores em reais (R$) e datas no formato brasileiro	Essencial
RNF21	Layout **responsivo**, utilizável em telas a partir de 375 px de largura	Importante
RNF22	Cores consistentes por status do pedido em todos os gráficos e tabelas	Desejável
RNF23	O link "Dashboard" só aparece no menu para os perfis com acesso	Importante

3.5 Manutenibilidade

ID	Requisito	Prioridade
RNF24	As regras de cálculo devem ficar em uma classe sem dependência de banco ou framework (`DashboardCalculadora`), para facilitar testes	Importante
RNF25	Arquitetura em camadas: repositório → serviço → controlador → apresentação	Essencial
RNF26	Regras de negócio e segurança devem ter **testes automatizados** (JUnit 5 + Mockito + MockMvc)	Importante
RNF27	Código e comentários em português, seguindo o padrão já adotado no projeto	Desejável

3.6 Compatibilidade e interoperabilidade

ID	Requisito	Prioridade
RNF28	Backend em **Java 21** e **Spring Boot 3.4**, com banco **MongoDB**	Essencial
RNF29	Datas trafegadas em formato **ISO 8601** (`AAAA-MM-DD`)	Essencial
RNF30	Erros da API no formato padrão do sistema: `{"erro": "mensagem"}`	Essencial

ID	Requisito	Prioridade
RNF31	O Grafana deve subir com **Docker Compose**, já com a fonte de dados e o dashboard configurados (*provisioning*), em versões fixas: Grafana 11.3.0 e plugin Infinity 3.7.1	Importante
RNF32	A tela deve funcionar nas versões atuais de Chrome, Edge e Firefox	Importante

4. Regras de negócio

ID	Regra

RN01	Pedido **cancelado não é venda**: não entra em faturamento, ticket médio, itens vendidos nem ranking. Entra no total de pedidos e na taxa de cancelamento
RN02	**Faturamento** usa o valor após o desconto autorizado pelo gerente
RN03	O desconto é distribuído **proporcionalmente** entre os itens: receita do item = subtotal × (1 − desconto%)
RN04	Com filtro de categoria, entram só os pedidos com algum item da categoria, e os valores somam apenas esses itens
RN05	O período anterior para comparação tem o **mesmo número de dias** do período consultado
RN06	O tempo médio de atendimento considera só os pedidos com data de saída registrada
RN07	Os valores "todos" (status) e "todas" (categoria) equivalem a não filtrar

5. Rastreabilidade

Requisito	Implementação	Teste automatizado
RF01, RN01, RN02	`DashboardCalculadora` (faturamento, ticketMedio...), `DashboardService.resumo`	`DashboardCalculadoraTest.indicadoresIgnoramPedidosCancelados`, `semPedidosNaoDividePorZero`
RF01.8, RN05	`DashboardService.resumo`, `variacao`	`DashboardServiceTest.variacaoPercentual`
RF02	`DashboardCalculadora.serieDeVendas`, `vendasPorHora`, `vendasPorCategoria`	`serieDiariaPreencheDiasSemVendaComZero`
RF03	`DashboardCalculadora.pedidosPorStatus`, `DashboardService.pedidosRecentes`	`statusTrazTodosOsStatusNaOrdemDoFluxo`
RF04	`DashboardCalculadora.produtosMaisVendidos`	`rankingPorQuantidadeSemCancelados`
RF05, RN07	`DashboardService.validar`, `dashboard.js` (`lerFiltros`)	`DashboardServiceTest.periodoPadraoSaoOsUltimos30Dias`, `normalizaStatusECategoria`, `todosETodasSignificamSemFiltro`, `rejeitaFiltrosInvalidos`; `DashboardCalculadoraTest.filtroDeStatus`
RN03, RN04	`DashboardCalculadora.linhasDo`	`filtroDeCategoriaSomaSoOsItensDaCategoriaComDescontoProporcional`
RF06	`DashboardApiController`, `DashboardDto`	`DashboardRotasSegurancaTest.gerenteEAdminAcessam`
RF07.6	`ApiExceptionHandler`	`DashboardRotasSegurancaTest.filtroInvalidoRetorna400ComMensagem`

Requisito	Implementação	Teste automatizado
RF07.8	`StatusPedido.de`, `DashboardCalculadora.linhasDo`	`pedidoAntigoDeProdutoUnicoEStatusAntigo`

6. Restrições e premissas

Restrições - O banco de dados é o MongoDB já usado pelo sistema (MongoDB Atlas), e o IP da máquina precisa estar liberado no Atlas. - O Grafana roda localmente via Docker; o Grafana Cloud não acessa o `localhost` sem um túnel público. - A chave da API (`DASHBOARD_API_KEY`) precisa ter o mesmo valor na aplicação e no Grafana.

Premissas - Os pedidos têm data de criação registrada; pedidos sem data não aparecem nos filtros por período. - A categoria de um item é a do cadastro atual do produto (campo `tipo`). - Nome e preço do item são os gravados no momento do pedido, então alterações posteriores no cadastro não mudam o histórico.

Documentação de referência dos endpoints que alimentam o Dashboard Administrativo (tela Grafana).

. Visão geral

Endpoints

Referente dos endpoints que alimentam o Dashboard Administrativo (tela Grafana).

1. Visão geral

Item	Valor
URL base	`http://localhost:8080/api/dashboard`
Métodos	Somente `GET` (API de leitura)
Formato	JSON, UTF-8
Datas	ISO 8601: `AAAA-MM-DD` (data) e `AAAA-MM-DDThh:mm:ss` (data e hora, horário local do servidor)
Valores em dinheiro	Número em reais com 2 casas decimais (ex.: `87.93` = R$ 87,93)
Percentuais	Número com 1 casa decimal (ex.: `4.1` = 4,1%)
Código-fonte	`controller/api/DashboardApiController.java`

Endpoint	Para quê	Requisito
`GET /resumo`	Indicadores gerais (cards)	RF01
`GET /vendas`	Evolução das vendas no tempo	RF02
`GET /vendas-por-hora`	Horário de pico	RF02
`GET /vendas-por-categoria`	Faturamento por categoria	RF02
`GET /status`	Pedidos por status	RF03
`GET /pedidos-recentes`	Últimos pedidos	RF03
`GET /produtos-mais-vendidos`	Ranking de produtos	RF04
`GET /filtros`	Opções para montar os filtros	RF05

2. Autenticação

A API aceita duas formas de autenticação.

2.1 Sessão do sistema (usada pela tela `/dashboard`)

Depois do login em `/login`, o navegador envia o cookie `JSESSIONID` automaticamente.

Perfil	Acesso
`GERENTE`	Permitido
`ADMIN`	Permitido
`ATENDENTE`	Negado (403)

2.2 Chave de API (usada pelo Grafana e por ferramentas externas)

Envie a chave no cabeçalho `X-API-KEY`:

```
GET /api/dashboard/resumo HTTP/1.1
Host: localhost:8080
X-API-KEY: <sua-chave>
```

- A chave é definida na variável de ambiente `DASHBOARD_API_KEY` da aplicação. Vazia = recurso desligado.
- Vale **somente** para `GET /api/dashboard/**`. Não dá acesso à tela, aos pedidos nem aos usuários.
- Não cria sessão: a chave precisa ir em toda requisição.

---

3. Parâmetros comuns (filtros)

Todos os endpoints, exceto `/filtros`, aceitam estes parâmetros de *query string*. Todos são opcionais.

Parâmetro	Tipo	Padrão	Regras	Exemplo
`inicio`	data `AAAA-MM-DD`	29 dias antes de `fim`	Não pode ser posterior a `fim`	`2026-11-01`
`fim`	data `AAAA-MM-DD`	hoje	Período total de no máximo **366 dias**	`2026-11-07`
`status`	texto	todos	Um status do pedido, sem diferença entre maiúsculas e minúsculas. `todos` = sem filtro. Aceita os nomes antigos `EM_PREPARO` e `ENTREGUE`	`FINALIZADO`
`categoria`	texto	todas	Uma categoria cadastrada (campo `tipo` do produto), sem diferença entre maiúsculas e minúsculas. `todas` = sem filtro	`Pizza`

Valores de `status`: `RECEBIDO`, `EM_PREPARACAO`, `PRONTO`, `SAIU_PARA_ENTREGA`, `RETIRADO`, `FINALIZADO`, `CANCELADO`.

Como o período funciona: `inicio` e `fim` são **inclusivos**: `fim=2026-11-07` inclui o dia 07 inteiro, até 23:59:59.

Como os filtros afetam os números: - **Pedidos cancelados** contam em `totalPedidos`, `pedidosCancelados` e `/status`, mas **não** contam como venda (faturamento, ticket, itens, ranking). - Com `categoria`, entram só os pedidos que têm algum item dessa categoria, e os valores somam **apenas os itens da categoria**, já com o desconto proporcional.

---

4. Endpoints

4.1 `GET /api/dashboard/resumo`

Indicadores gerais do período e comparação com o período anterior de mesmo tamanho.

Parâmetros: os comuns.

Exemplo de requisição

```
curl -H "X-API-KEY: <sua-chave>" "http://localhost:8080/api/dashboard/resumo?inicio=2026-11-01&fim=2026-11-07"
```

Resposta `200 OK`


```
{
  "filtro": {
    "inicio": "2026-11-01",
    "fim": "2026-11-07",
    "status": null,
    "categoria": null
  },
  "faturamento": 30510.59,
  "totalPedidos": 362,
  "pedidosValidos": 347,
  "pedidosCancelados": 15,
  "ticketMedio": 87.93,
  "taxaCancelamento": 4.1,
  "itensVendidos": 1375,
  "totalDescontos": 144.41,
  "tempoMedioSaidaMinutos": 34.2,
  "comparativo": {
    "inicioAnterior": "2026-10-25",
    "fimAnterior": "2026-10-31",
    "faturamentoAnterior": 33437.21,
    "pedidosAnteriores": 389,
    "variacaoFaturamento": -8.8,
    "variacaoPedidos": -10.8
  }
}
```


Campos

Campo	Tipo	Descrição
`filtro`	objeto	Filtros que foram **efetivamente aplicados**, já com os valores padrão e normalizados (ex.: `em_preparo` → `EM_PREPARACAO`). `null` = sem filtro
`faturamento`	número (R$)	Soma da receita líquida (após desconto) dos pedidos não cancelados
`totalPedidos`	inteiro	Todos os pedidos do período, inclusive cancelados
`pedidosValidos`	inteiro	Pedidos não cancelados
`pedidosCancelados`	inteiro	Pedidos com status `CANCELADO`
`ticketMedio`	número (R$)	`faturamento ÷ pedidosValidos` (0 se não houver pedidos)
`taxaCancelamento`	número (%)	`pedidosCancelados ÷ totalPedidos × 100`
`itensVendidos`	inteiro	Unidades vendidas nos pedidos válidos
`totalDescontos`	número (R$)	Valor concedido em descontos nos pedidos válidos
`tempoMedioSaidaMinutos`	número ou `null`	Média, em minutos, entre a entrada do pedido e a saída (entrega ou retirada). `null` se nenhum pedido do período tiver saída registrada

`comparativo.inicioAnterior` / `fimAnterior`	data	Período anterior usado na comparação (mesmo número de dias)
`comparativo.faturamentoAnterior`	número (R$)	Faturamento do período anterior
`comparativo.pedidosAnteriores`	inteiro	Pedidos válidos do período anterior
`comparativo.variacaoFaturamento`	número (%) ou `null`	`(atual − anterior) ÷ anterior × 100`. `null` se o anterior for zero
`comparativo.variacaoPedidos`	número (%) ou `null`	Mesma fórmula, para pedidos válidos

Erros possíveis: 400 de filtro, 401, 403.

4.2 `GET /api/dashboard/vendas`

Série temporal de faturamento e pedidos, para o gráfico de evolução:

Parâmetro	Tipo	Padrão	Valores
`agrupamento`	texto	`DIA`	`DIA`, `SEMANA` (começa na segunda-feira) ou `MES` (começa no dia 1)

Exemplo de requisição

```
curl -H "X-API-KEY: <sua-chave>" "http://localhost:8080/api/dashboard/vendas?inicio=2026-11-01&fim=2026-11-07&agrupamento=DIA"
```

Resposta `200 OK`

```
[
  { "periodo": "2026-11-01", "rotulo": "01/11", "faturamento": 5379.55, "pedidos": 66 },
  { "periodo": "2026-11-02", "rotulo": "02/11", "faturamento": 3464.93, "pedidos": 40 },
  { "periodo": "2026-11-03", "rotulo": "03/11", "faturamento": 3646.22, "pedidos": 40 },
  "..."
]
```

Campo	Tipo	Descrição
`periodo`	data	Início do período (o dia, a segunda-feira da semana ou o dia 1 do mês)
`rotulo`	texto	Texto pronto para o eixo do gráfico: `01/11` (dia), `Sem. 02/11` (semana), `nov./26` (mês)
`faturamento`	número (R$)	Faturamento do período (sem cancelados)
`pedidos`	inteiro	Pedidos válidos do período

4.3 `GET /api/dashboard/vendas-por-hora`

Pedidos e faturamento por hora do dia, para identificar o horário de pico.

Exemplo de requisição

```
curl -H "X-API-KEY: <sua-chave>" "http://localhost:8080/api/dashboard/vendas-por-hora?inicio=2026-11-01&fim=2026-11-07"
```

Resposta `200 OK`: sempre **24 itens**, da hora 0 à 23.

```
[
  { "hora": 0, "faturamento": 0.0, "pedidos": 0 },
  { "hora": 1, "faturamento": 0.0, "pedidos": 0 },
  "...",
  { "hora": 23, "faturamento": 0.0, "pedidos": 0}
]
```

Campo	Tipo	Descrição
`hora`	inteiro (0–23)	Hora de entrada do pedido (ex.: `20` = das 20:00 às 20:59)
`faturamento`	número (R$)	Faturamento dos pedidos válidos daquela hora
`pedidos`	inteiro	Quantidade de pedidos válidos daquela hora

Erros possíveis: 400 de filtro, 401, 403.

4.4 `GET /api/dashboard/vendas-por-categoria`

Faturamento e unidades vendidas por categoria de produto, do maior para o menor faturamento. **Parâmetros:** os comuns.

Exemplo de requisição

```
curl -H "X-API-KEY: <sua-chave>" "http://localhost:8080/api/dashboard/vendas-por-categoria?inicio=2026-11-01&fim=2026-11-07"
```

Resposta `200 OK`

```
[
  { "categoria": "Pizza",     "faturamento": 22021.89, "quantidade": 433 },
  { "categoria": "Esfiha",    "faturamento": 4655.76,  "quantidade": 694 },
  { "categoria": "Bebida",    "faturamento": 1771.4,   "quantidade": 169 },
  { "categoria": "Lanche",    "faturamento": 1663.51,  "quantidade": 56 },
  { "categoria": "Sobremesa", "faturamento": 398.04,   "quantidade": 23}
]
```

Campo	Tipo	Descrição
`categoria`	texto	Campo `tipo` do produto. Produtos sem tipo aparecem como `Sem categoria`
`faturamento`	número (R$)	Receita líquida dos itens da categoria
`quantidade`	inteiro	Unidades vendidas

4.5 `GET /api/dashboard/status`

Quantidade de pedidos em cada status.

Parâmetros: os comuns.

Exemplo de requisição

```
curl -H "X-API-KEY: <sua-chave>" "http://localhost:8080/api/dashboard/status?inicio=2026-11-01&fim=2026-11-07"
```

Resposta `200 OK` : sempre os **7 status**, na ordem do fluxo do pedido, mesmo os que estão com zero.

```
[
  { "status": "RECEBIDO",          "descricao": "Recebido",          "quantidade": 0,   "percentual" 0.0 },
  { "status": "EM_PREPARACAO",     "descricao": "Em preparação",     "quantidade": 0,   "percentual" 0.0 },
  { "status": "PRONTO",            "descricao": "Pronto",            "quantidade": 0,   "percentual" 0.0 },
  { "status": "SAIU_PARA_ENTREGA", "descricao": "Saiu para entrega", "quantidade": 0,   "percentual" 0.0 },
  { "status": "RETIRADO",          "descricao": "Retirado",          "quantidade": 0,   "percentual" 0.0 },
  { "status": "FINALIZADO",        "descricao": "Finalizado",        "quantidade": 347, "percentual" 95.9 },
  { "status": "CANCELADO",         "descricao": "Cancelado",         "quantidade": 15,  "percentual" 4.1 }
]
```

Campo	Tipo	Descrição
`status`	texto	Código do status (use como valor do filtro `status`)
`descricao`	texto	Nome para exibir na tela
`quantidade`	inteiro	Pedidos nesse status, **incluindo cancelados**
`percentual`	número (%)	Participação no total de pedidos do período

Erros possíveis: 400 de filtro, 401, 403.

4.6 `GET /api/dashboard/produtos-mais-vendidos`

Ranking de produtos por quantidade vendida (o desempate é pelo faturamento). Itens de pedidos cancelados não entram.

Parâmetros: os comuns e mais:

Parâmetro	Tipo	Padrão	Regras
`limite`	inteiro	`5`	De 1 a 50

Exemplo de requisição

```
curl -H "X-API-KEY: <sua-chave>" "http://localhost:8080/api/dashboard/produtos-mais-vendidos?inicio=2026-11-01&fim=2026-11-07&limite=3"
```

Resposta `200 OK`

Campo	Tipo	Descrição
`posicao`	inteiro	Posição no ranking (1 = mais vendido)
`produtoId`	texto ou `null`	ID do produto. `null` em pedidos antigos gravados sem ID de produto
`nome`	texto	Nome do produto **no momento da venda**
`categoria`	texto	Categoria atual do produto

`quantidade`	inteiro	Unidades vendidas no período
`faturamento`	número (R$)	Receita líquida do produto no período

Erros possíveis: 400 de filtro, `400` com limite inválido, 401, 403.

4.7 `GET /api/dashboard/pedidos-recentes`

Últimos pedidos do período, do mais recente para o mais antigo (inclui cancelados).

Parâmetros: os comuns e mais:

Parâmetro	Tipo	Padrão	Regras
`limite`	inteiro	`10`	De 1 a 50

Exemplo de requisição

```
curl -H "X-API-KEY: <sua-chave>" "http://localhost:8080/api/dashboard/pedidos-recentes?inicio=2026-11-01&fim=2026-11-07&limite=2"
```

Resposta `200 OK`

```
[
  {
    "id": "12917",
    "dataCriacao": "2026-11-07T14:48:33",
    "cliente": "André Ferreira",
    "itens": "8x Esfiha de Carne",
    "status": "FINALIZADO",
    "descricaoStatus": "Finalizado",
    "valor": 52.0
  },
  {
    "id": "12924",
    "dataCriacao": "2026-11-07T14:08:23",
    "cliente": "Lucas Silva",
    "itens": "1x Coca-Cola 2L, 1x Cerveja Lata",
    "status": "FINALIZADO",
    "descricaoStatus": "Finalizado",
    "valor": 18.9
  }
]
```

Campo	Tipo	Descrição

`id`	texto	ID do pedido (abre em `/pedidos/{id}` no sistema)
`dataCriacao`	data e hora	Entrada do pedido
`cliente`	texto	Nome do cliente, ou `Cliente não identificado` se o cadastro não existir
`itens`	texto	Itens resumidos, separados por vírgula (`2x Pizza Calabresa (borda recheada)`)
`status`	texto	Código do status
`descricaoStatus`	texto	Nome do status para exibição (`Desconhecido` se o valor gravado for inválido)
`valor`	número (R$)	Valor líquido do pedido. Com filtro de `categoria` , mostra só os itens da categoria

Erros possíveis: 400 de filtro, `400` com limite inválido, 401, 403.

4.8 `GET /api/dashboard/filtros`

Opções disponíveis para montar a tela de filtros.

Parâmetros: nenhum.

Exemplo de requisição

```
curl -H "X-API-KEY: <sua-chave>" "http://localhost:8080/api/dashboard/filtros"
```

Resposta `200 OK`

```
{
  "status": [
    { "valor": "RECEBIDO", "descricao": "Recebido" },
    { "valor": "EM_PREPARACAO", "descricao": "Em preparação" },
    { "valor": "PRONTO", "descricao": "Pronto" },
    { "valor": "SAIU_PARA_ENTREGA", "descricao": "Saiu para entrega" },
    { "valor": "RETIRADO", "descricao": "Retirado" },
    { "valor": "FINALIZADO", "descricao": "Finalizado" },
    { "valor": "CANCELADO", "descricao": "Cancelado"}
  ],
  "categorias": ["Bebida", "Esfiha", "Lanche", "Pizza", "Sobremesa"],
  "inicioPadrao": "2026-10-09",
  "fimPadrao": "2026-11-07",
  "diasMaximos": 366
}
```

Campo	Tipo	Descrição
	lista	Status possíveis: `valor` vai no filtro e `descricao` aparece na tela
`categorias`	lista de texto	Categorias dos produtos cadastrados, em ordem alfabética
`status`	data	Período usado quando nenhuma data é informada (últimos 30 dias)
`inicioPadrao` / `fimPadrao`		
`diasMaximos`	inteiro	Tamanho máximo do período aceito pela API

Erros possíveis: 401, 403.

---

5. Erros

5.1 Formato e códigos

Os erros da aplicação seguem o padrão do sistema:

```
{ "erro": "mensagem explicando o problema"}
```

Por segurança, a resposta nunca traz detalhes técnicos (stack trace, nomes de classes ou consultas).

Código	Quando acontece	Corpo da resposta
`200 OK`	Sucesso	Os dados do endpoint
`400 Bad Request`	Filtro ou parâmetro inválido	`{"erro": "..."}` (ver 5.2)
`401 Unauthorized`	Sem login, sessão expirada, chave ausente ou errada	`{"erro": "Autenticação necessária."}`
`403 Forbidden`	Usuário logado sem permissão (ex.: ATENDENTE)	`{"erro": "Você não tem permissão para esta ação."}`
`403 Forbidden`	Método diferente de GET (`POST` , `PUT` , `DELETE`)	`{"erro": "Token CSRF ausente ou inválido. Envie o cabeçalho X-CSRF-TOKEN."}`
`404 Not Found`	Endpoint inexistente	Formato padrão do Spring Boot (ver abaixo)
`500 Internal Server Error`	Falha inesperada (ex.: banco de dados fora do ar)	Formato padrão do Spring Boot, sem detalhes

Exemplo de `404` (formato padrão do Spring Boot, diferente do `{"erro"}`):

5.2 Erros 400 detalhados

Todas as mensagens abaixo foram obtidas da API real.

Situação	Exemplo de requisição	Resposta
Data inicial depois da final	`/resumo?inicio=2026-11-07&fim=2026-11-01`	`{"erro":"A data inicial não pode ser posterior à data final."}`
Período maior que 366 dias	`/resumo?inicio=2025-01-01&fim=2026-11-07`	`{"erro":"O período máximo de consulta é de 366 dias."}`
Data em formato errado	`/resumo?inicio=07/11/2026`	`{"erro":"Requisição inválida: verifique os parâmetros e o corpo enviados."}`
Status inexistente	`/resumo?status=ENTREGANDO`	`{"erro":"Status inválido: ENTREGANDO"}`
Categoria inexistente	`/resumo?categoria=Sushi`	`{"erro":"Categoria inválida: Sushi"}`
Agrupamento inválido	`/vendas?agrupamento=ANO`	`{"erro":"Agrupamento inválido. Use DIA, SEMANA ou MES."}`
Limite fora da faixa	`/produtos-mais-vendidos?limite=0`	`{"erro":"O limite deve estar entre 1 e 50."}`
Limite não numérico	`/produtos-mais-vendidos?limite=abc`	`{"erro":"Requisição inválida: verifique os parâmetros e o corpo enviados."}`

5.3 Como o cliente deve tratar os erros

É o que a tela `/dashboard` faz (`static/js/dashboard.js`):

Código	Ação recomendada

`400`	Mostrar ao usuário a mensagem do campo `erro`
`401`	Redirecionar para o login (`/login?expirada`)
`403`	Mostrar "Você não tem permissão para ver o dashboard."
Falha de rede	Mostrar "Não foi possível conectar ao servidor." e oferecer "Tentar novamente"
`5xx`	Mostrar mensagem genérica e oferecer "Tentar novamente"

---

6. Testando com curl, Postman e navegador

curl (PowerShell)

No PowerShell, use `curl.exe` (o `curl` sozinho é um apelido de outro comando):

```
$chave = (Get-Content grafana\.env) -replace '^DASHBOARD_API_KEY=', ''
curl.exe -H "X-API-KEY: $chave" "http://localhost:8080/api/dashboard/resumo?inicio=2026-11-01&fim=2026-11-07"
```

Postman / Insomnia

1. Método **GET** e URL `http://localhost:8080/api/dashboard/resumo` .
2. Aba **Params**: `inicio` , `fim` , `status` , `categoria` (todos opcionais).
3. Aba **Headers**: `X-API-KEY` com o valor da chave.

Navegador

Faça login no sistema como `gerente` e acesse a URL direto, por exemplo `http://localhost:8080/api/dashboard/resumo` . O cookie de sessão é usado, sem precisar de chave.

Evidências de testes realizados

---

1. Resumo da entrega

O `DashboardApiController` passou a ter **23 testes de integração ponta a ponta**. Em cada um, a aplicação Spring Boot sobe inteira, recebe requisições HTTP reais e consulta um **MongoDB 7 real**, criado em Docker só para o teste.

Esses testes garantem que os 8 endpoints: - calculam os indicadores corretamente a partir do que está gravado no banco; - respeitam os limites do período e os filtros; - tratam dados antigos; - devolvem os erros padronizados; - protegem o acesso pela chave `X-API-KEY` ; - não alteram nada no banco.

Para completar, há **7 testes de segurança com MockMvc** e **21 testes unitários** das regras de cálculo, da validação de filtros e do gerador de dados de demonstração.

---

2. Configuração do ambiente de teste

Item	Configuração
Estratégia	**Ambos.** `@SpringBootTest(webEnvironment = RANDOM_PORT)` + `TestRestTemplate` para a integração ponta a ponta; `@WebMvcTest` + `MockMvc` com o `SecurityConfig` real para regras de segurança isoladas
Banco de teste	**Testcontainers + imagem `mongo:7.0`** (MongoDB real e descartável em Docker, *replica set* de 1 nó). Banco `pediupatrao_teste`
Arquivo de configuração	Nenhum arquivo separado. As propriedades de teste ficam na própria classe: `@SpringBootTest(properties = ...)` define a chave da API e desliga usuários e dados de demonstração, e `@DynamicPropertySource` aponta `spring.data.mongodb.uri` para o container, sobrescrevendo a URL do Atlas do `application.properties`
Proteção do banco	O teste `ambiente_usaMongoDoContainerENaoOAtlas` falha se a URI de conexão contiver `mongodb.net` **de produção**
Isolamento de dados	Em `@BeforeEach` , as coleções `pedidos` , `produtos` e `clientes` são apagadas e recriadas com uma massa fixa (IDs `P1` … `P7` , `c1` , `p-calab` …). As datas ficam numa semana fixa (01 a 07/11/2026) e toda consulta informa `inicio` / `fim` , então **o resultado não depende do dia em que o teste roda**

Item	Configuração
Sem Docker	`@Testcontainers(disabledWithoutDocker = true)` : pelo comportamento documentado do Testcontainers, a classe de integração é **ignorada** em vez de falhar. Não foi testado aqui, porque o Docker estava ligado na execução
Componentes mockados e motivo	**Integração (`DashboardApiIntegrationTest`): nenhum.** Slice de segurança (`DashboardRotasSegurancaTest`): `DashboardService` , para testar só as regras de acesso sem banco; `AuditoriaService` e `UsuariDetailsService` , que são dependências do `SecurityConfig` . Unitários: repositórios simulados com Mockito
Ferramentas	JUnit 5, Spring Boot Test 3.4.4, Spring Security Test, Mockito, Testcontainers (versões gerenciadas pelo Spring Boot), Jackson
Ambiente de execução	Windows 11, Java 21, Maven 3.9.9 (via `mvnw`), Docker Engine 24.0.7 (Docker Desktop)

Massa de dados da integração

Pedido	Data/hora	Status	Itens	Valor	Papel no teste
P1	01/11 19:30	FINALIZADO	2x Calabresa (50) + 1x Refri (10), desconto 10%	R$ 99,00	Desconto, tempo de saída (30 min)
P2	03/11 20:10	EM_PREPARACAO	1x Mussarela (45)	R$ 45,00	Cliente inexistente no cadastro
P3	03/11 21:00	CANCELADO	5x Calabresa	R$ 250,00	Não pode contar como venda
P5	07/11 23:59:59	FINALIZADO	1x Refri	R$ 10,00	Último segundo do período: **entra**
P6	08/11 00:00:00	FINALIZADO	1x Calabresa	R$ 50,00	Dia seguinte: **não entra**
P7	28/10 20:00	FINALIZADO	2x Calabresa	R$ 100,00	Período anterior (comparativo)
P8	05/11 12:00	`ENTREGUE` (nome antigo)	formato antigo, 3x Calabresa	R$ 120,00	Inserido só no teste de dados antigos

Resultado esperado para 01–07/11: faturamento **R$ 154,00** (99 + 45 + 10), 4 pedidos, 3 válidos, 1 cancelado, ticket médio **R$ 51,33** e taxa de cancelamento **25%**.

3. Cenários de integração implementados

Arquivo: `src/test/java/com/umc/pediupatrao/integration/DashboardApiIntegrationTest.java`

#	Endpoint	Cenário	Tipo	Status	Verificado além do status	Método de teste
1	—	A aplicação usa o banco do container, não o Atlas	Persistência	—	URI de conexão sem `mongodb.net`; container ativo	`ambiente_usaMongoDoContainerENaoOAtlas`
2	GET `/resumo`	Semana com dados	Feliz	200	faturamento 154, total 4, válidos 3, cancelados 1, ticket 51,33, taxa 25%, itens 5, descontos 11, tempo médio 30 min, filtro aplicado	`resumo_semanaComDados_deveCalcularIndicadoresAPartirDoBanco`

#	Endpoint	Cenário	Tipo	Status	Verificado além do status	Método de teste
3	GET `/resumo`	Comparação com o período anterior	Feliz	200	período 25–31/10, faturamento anterior 100, variação +54% e +200%	`resumo_deveCompararComPeriodoAnteriorDeMesmoTamanho`
4	GET `/resumo`	Limites do período (23:59:59 entra, 00:00 do dia seguinte não)	Persistência	200	total 1, faturamento 10	`resumo_limitesDoPeriodo_incluiUltimoSegundoDoFimEExcluiDiaSeguinte`
5	GET `/resumo`	Pedido gravado no formato antigo, com status `ENTREGUE`	Persistência	200	faturamento 120, itens 3	`resumo_pedidoAntigoComStatusAntigo_deveSerConvertido`
6	GET `/resumo`	Filtro `categoria=bebida` (minúsculo)	Feliz	200	total 2, faturamento 19 (com desconto proporcional), categoria `filtroCategoria` normalizada para `Bebida`	`categoriaSoSomaItensDaCategoriaComDescontoProporcional`
7	GET `/resumo`	Filtro `status=em_preparo` (nome antigo)	Feliz	200	status normalizado `EM_PREPARACAO`, total 1, faturamento 45	`filtroStatus_comNomeAntigo_deveSerNormalizado`
8	GET `/filtros`	Opções de filtro	Relacionamento	200	categorias lidas dos produtos do banco (`Bebida` , `Pizza`), 7 status, máximo de 366 dias	`filtros_deveListarCategoriasCadastradasNoBanco`
9	GET `/vendas`	Agrupamento por dia	Feliz	200	7 pontos, rótulo `01/11` , dia sem venda = 0, cancelado fora	`vendasPorDia_devePreencherDiasSemVendaComZero`
10	GET `/vendas`	Agrupamento por mês	Feliz	200	2 pontos (out = 100, nov = 154)	`vendasPorMes_deveAgruparPorMes`
11	GET `/vendas-por-hora`	Distribuição por hora	Feliz	200	24 posições; pedidos nas horas 19, 20 e 23; cancelado das 21h não conta	`vendasPorHora_deveTer24PosicoesComOsPedidosNaHoraCerta`
12	GET `/vendas-por-categoria`	Faturamento por categoria	Feliz	200	ordem Pizza (135) → Bebida (19)	`vendasPorCategoria_deveOrdenarPorFaturamento`
13	GET `/status`	Pedidos por status	Feliz	200	7 status na ordem do fluxo; EM_PREPARACAO 1, FINALIZADO 2, CANCELADO 1	`status_deveContarTodosOsStatusInclusiveCancelado`
14	GET `/pedidos-recentes`	Últimos pedidos	Relacionamento	200	ordem decrescente, nome do cliente vindo da coleção `clientes` , "Cliente não identificado", descrição dos itens	`pedidosRecentes_deveOrdenarDoMaisNovoEResolverNomeDoCliente`
15	GET `/produtos-mais-vendidos`	Ranking	Relacionamento	200	ordem Calabresa → Refri → Mussarela; cancelado fora; desempate pela receita; categoria vinda do produto	`produtosMaisVendidos_deveRankearPorQuantidadeSemCancelados`

| # | Endpoint | Cenário | Tipo | Status | Verificado além do status | Método de teste |

16	GET `/resumo`	Data inicial depois da final	Erro	400	mensagem exata em `erro`	`dataInicialMaiorQueFinal_deveRetornar400ComMensagem`
17	GET `/resumo`	Categoria que não existe no banco	Erro	400	`"Categoria inválida: Sushi"`	`categoriaInexistenteNoBanco_deveRetornar400`
18	GET `/produtos-mais-vendidos`	`limite=51`	Erro	400	—	`limiteForaDaFaixa_deveRetornar400`
19	GET `/resumo`	Sem chave e sem login	Segurança	401	—	`semChave_deveRetornar401`
20	GET `/resumo`	Chave errada	Segurança	401	—	`chaveErrada_deveRetornar401`
21	GET `/api/pedidos`, `/api/usuarios`	A chave do dashboard não abre outras APIs	Segurança	401	—	`chaveNaoDaAcessoAOutrasApis`
22	POST / DELETE `/resumo`	Tentativa de escrita	Segurança / Persistência	403	quantidade de pedidos no banco igual antes e depois	`apiSomenteLeitura_postENegadoEBancoNaoMuda`
23	—	Índice criado na inicialização	Persistência	—	índice `idx_dataCriacao` existe na coleção `pedidos`	`inicializacao_deveCriarIndiceDeDataNosPedidos`

Total de cenários de integração: 23 (Feliz: 9 · Erro: 3 · Segurança: 4 · Persistência: 4 · Relacionamento: 3, contando o cenário 22 como Segurança)

3.1 Cenários complementares de segurança (MockMvc + SecurityConfig real)

Arquivo: `src/test/java/com/umc/pediupatrao/security/DashboardRotasSegurancaTest.java`

Estes testes cobrem situações com **perfis de usuário logado**, que no teste ponta a ponta exigiriam simular o login por formulário.

#	Endpoint	Cenário	Tipo	Status	Verificado além do status	Método de teste
24	GET `/api/dashboard/resumo`	Anônimo	Segurança	401	—	`anonimoRecebe401`
25	GET `/api/dashboard/resumo`, `/dashboard`	Perfil ATENDENTE	Segurança	403	tela e API negadas	`atendenteNaoAcessaODashboard`
26	GET `/api/dashboard/status`	Perfis GERENTE e ADMIN	Segurança	200	—	`gerenteEAdminAcessam`
27	GET `/api/dashboard/status`	Chave correta	Segurança	200	—	`grafanaComChaveCorretaLeOResumo`
28	GET `/api/dashboard/status`	Chave errada	Segurança	401	—	`chaveErradaOuAusenteRecebe401`
29	GET `/dashboard`, `/api/usuarios`	A chave não abre a tela nem outras APIs	Segurança	302 / 401	redireciona para o login	`chaveNaoAbreATelaNemOutrasApis`

| # | Endpoint | Cenário | Tipo | Status | Verificado além do status | Método de teste |

30	GET	`/api/dashboard/re sumo`	Filtro inválido e data em formato errado em `filtroInvalidoRetorn a400ComMensagem` e retorno de erro 400 com mensagem

3.2 Testes unitários do módulo

Arquivo	Qtd.	O que garante
`DashboardCalculadoraTest`	8	Cancelado não conta; desconto proporcional por categoria; filtro por status; ranking; série com dias zerados; todos os status; sem divisão por zero; pedidos e status antigos
`DashboardServiceTest`	5	Período padrão de 30 dias; normalização de status e categoria; `todos` / `todas` ; rejeição de filtros inválidos; cálculo da variação
`GeradorDadosDemoTest`	8	Volume de médio porte; nenhum pedido no futuro; IDs ; valores coerentes; fluxo de status válido; taxa de cancelamento; fim de semana > segunda; reaproveita produto existente; mesma semente gera os mesmos dados

5. Como executar os testes

Pré-requisito para os testes de integração: **Docker Desktop aberto**. Na primeira execução, a imagem `mongo:7.0` é baixada automaticamente.

Todos os testes:

```
./mvnw clean test
```

No PowerShell:

```
.\mvnw.cmd clean test
```

Somente os testes de integração:

```
./mvnw test -Dtest=DashboardApiIntegrationTest
```

Somente os testes do dashboard:

```
./mvnw test -Dtest="Dashboard*Test,GeradorDadosTest"
```

Os relatórios ficam em `target/surefire-reports/`.

6. Evidências

6.1 Resultado da execução automatizada

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.009 s -- in
com.umc.pediupatrao.entity.PedidoTest
[INFO] Tests run: 23, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 15.57 s -- in
com.umc.pediupatrao.integration.DashboardApiIntegrationTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.677 s -- in
com.umc.pediupatrao.security.DashboardRotasSegurancaTest
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.562 s -- in
com.umc.pediupatrao.security.UsuarioRotasSegurancaTest
```

6.2 Testes manuais e exploratórios

Além dos testes automatizados, foram feitas estas verificações no sistema real:

#	O que foi verificado	Como	Resultado
M1	API com o banco real (MongoDB Atlas)	`curl` com `X-API-KEY` nos 8 endpoints	Todos responderam 200 com dados coerentes. As respostas estão em `docs/API.md`
M2	Os 8 erros 400 e os erros 401/403/404	`curl` com parâmetros inválidos, sem chave, com chave errada, `POST` e login como atendente	Mensagens exatamente como documentado em `docs/API.md` , seção 5
M3	Tela `/dashboard` (cards, gráficos, tabelas, filtros)	Navegador, com o JSON real da API	Valores exibidos corretamente (ex.: faturamento R$ 13.533,00, ranking, últimos pedidos)
M4	Grafana ponta a ponta	Docker Compose + navegador em `localhost:3000`	17 painéis carregados com os dados reais (R$ 316 antes da carga de demonstração)
M5	Filtros do Grafana enviados	Log das requisições recebidas	`inicio` , `fim` , `status=todos` , `categoria=` para a API `todas` , `agrupamento` e `limite` corretos
M6	Desempenho com cerca de 13 mil pedidos	`curl` medindo `time_total` , consulta de 8 meses	**Antes: ~4,5 s → depois do índice e da projeção: ~1,1 s.** Consulta padrão de 30 dias: ~0,6 s

Testes de integração exigem Docker	O Testcontainers precisa do Docker para criar o MongoDB	`disabledWithoutDocker = true` : sem Docker, a classe é ignorada e a suíte não quebra

Dashboard administrativo funcional

Guia de uso para o **gerente** ou o **responsável pelo estabelecimento**: o que cada parte da tela mostra, como usar os filtros e como transformar os números em decisões.

1. Para que serve

O dashboard reúne numa única tela informações do dia a dia da loja:

- Faturamento, pedidos e ticket médio.
- Comparação com o período anterior.
- Dias e horários de pico.
- Produtos e categorias campeãs.
- Pedidos em cada etapa, cancelamentos e tempo de atendimento.

---

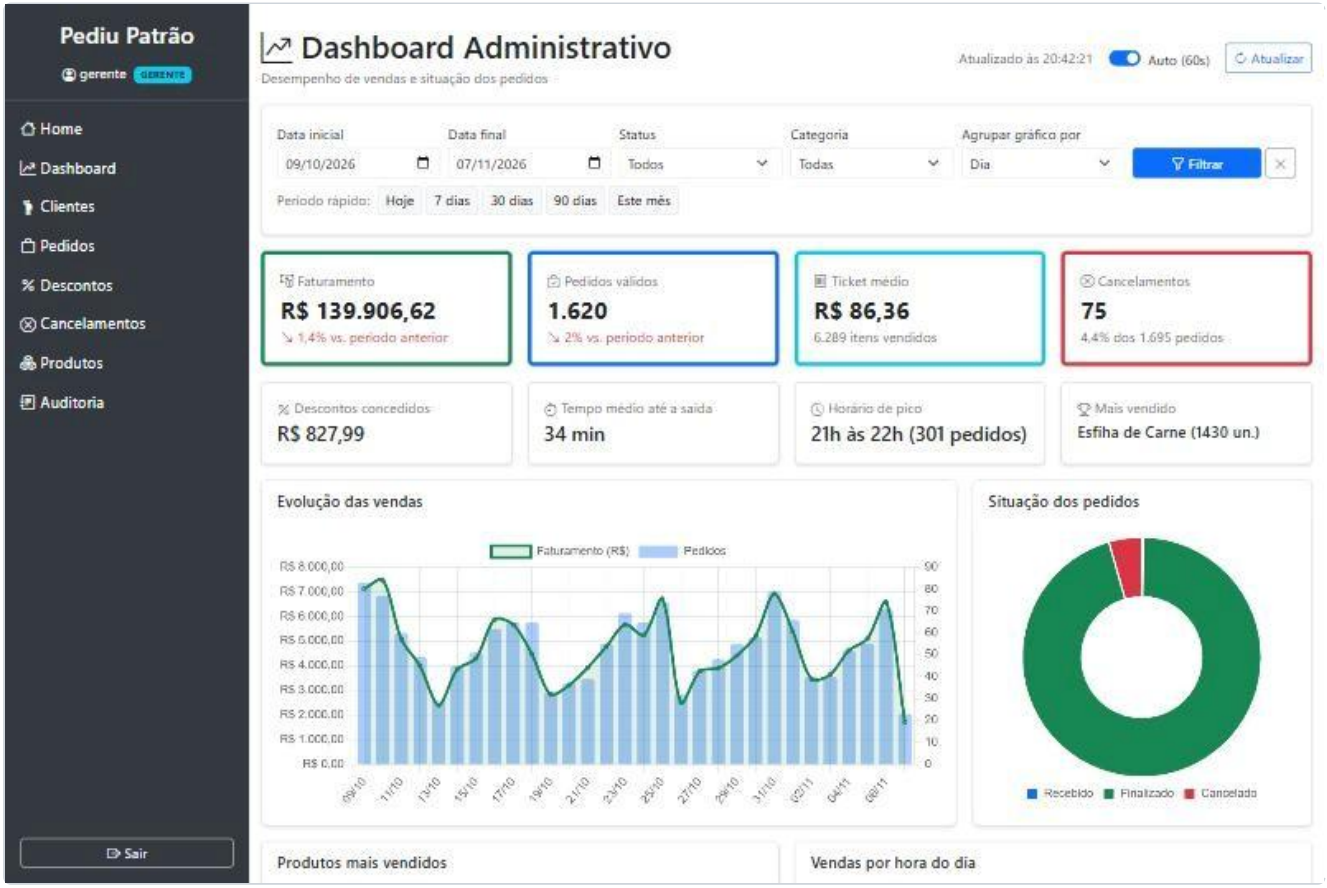
2. Como acessar

1. Entre no sistema (`http://localhost:8080`) com um usuário **Gerente** ou **Administrador**.
2. No menu lateral, clique em **Dashboard**.

Perfil	Vê o dashboard?
Gerente	Sim
Administrador	Sim
Atendente	Não. O item nem aparece no menu, e o acesso direto pela URL é bloqueado

---

3. Visão geral da tela



A tela é organizada de cima para baixo:

Área	Descrição
Cabeçalho	Título, horário da última atualização, chave **Auto (60s)** e botão **Atualizar**

Filtros	Período, status, categoria, agrupamento do gráfico e atalhos de período
Indicadores	8 cartões com os números principais
Gráficos	Evolução das vendas e situação dos pedidos
Análises	Produtos mais vendidos, vendas por hora e faturamento por categoria
Últimos pedidos	Tabela com os pedidos mais recentes do período

Ao abrir, o dashboard mostra os **últimos 30 dias**.

4. Indicadores

Os cartões do topo resumem o período escolhido. Na imagem acima, o período é de 09/10 a 07/11/2026.

Linha principal

Cartão	Descrição	Exemplo da tela	Explicação
Faturamento	Total vendido, já descontados os descontos. **Não inclui pedidos cancelados**	R$ 139.906,62	A seta mostra a variação em relação ao período anterior de mesmo tamanho (verde = subiu, vermelho = caiu)
Pedidos válidos	Pedidos do período, sem os cancelados	1.620 (↓ 2%)	Mede o volume de vendas, independente do valor

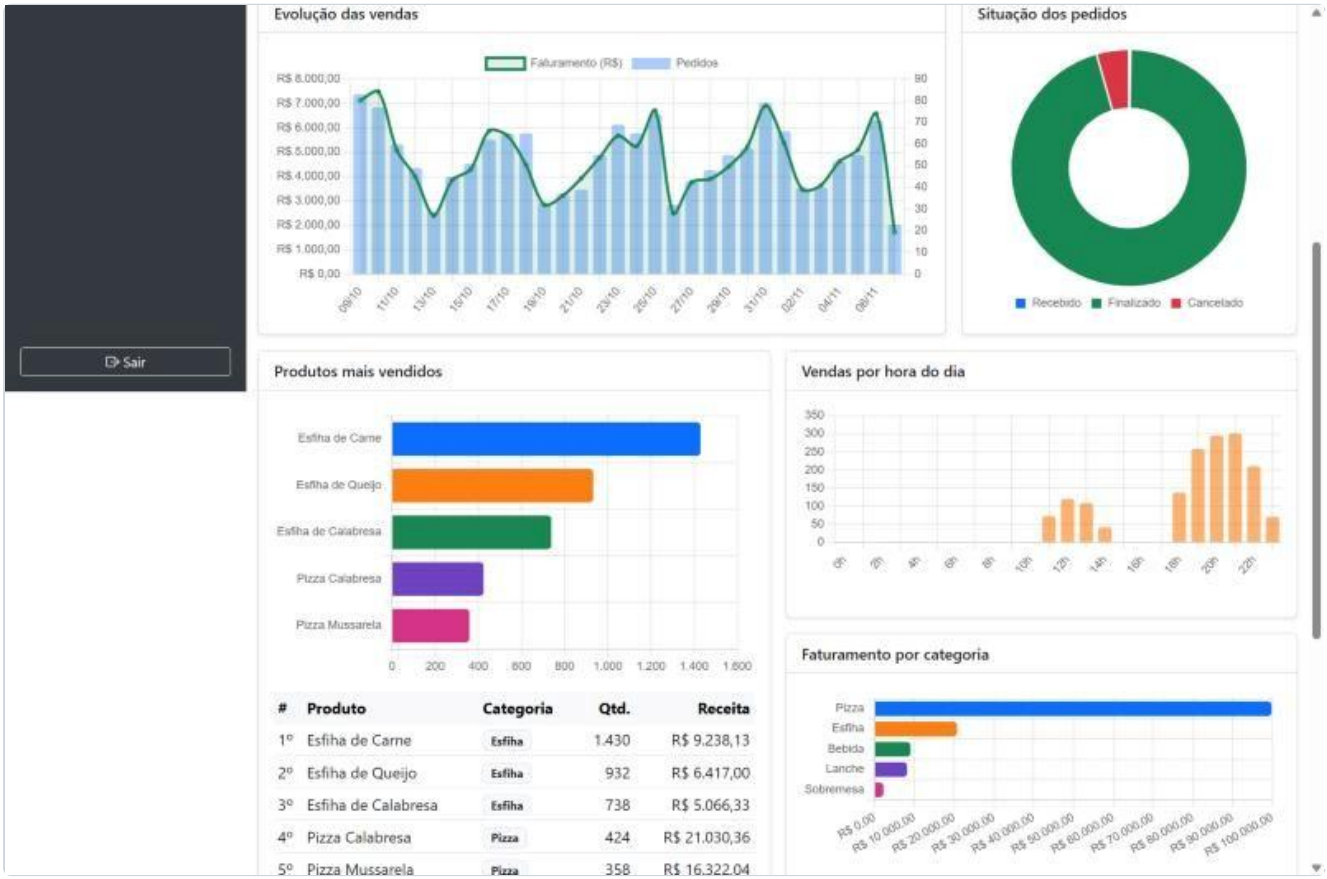
Cartão	Descrição	Exemplo da tela	Explicação
Ticket médio	Quanto cada pedido vale em média (faturamento ÷ pedidos válidos)	R$ 86,36 · 6.289 itens vendidos	Ticket subindo indica clientes comprando mais por pedido (combos, bebidas, sobremesas)
Cancelamentos	Pedidos cancelados e a sobre o total	75 · 4,4% dos 1.695 pedidos	Acima de 5% merece investigação (ver seção 10)

Linha secundária

Cartão	O que significa	Exemplo da tela
Descontos concedidos	Soma dos descontos autorizados pelo gerente	R$ 827,99

Tempo médio até a saída	Tempo entre a entrada do pedido e a saída para entrega ou a retirada no balcão	34 min
Horário de pico	Hora do dia com mais pedidos no período	21h às 22h (301 pedidos)
Mais vendido	Produto com mais unidades vendidas	Esfiha de Carne (1.430 un.)

5. Gráficos



#	Produto	Categoria	Qtd.	Receita
1º	Esfiha de Carne	Esfiha	1.430	R$ 9.238,13
2º	Esfiha de Queijo	Esfiha	932	R$ 6.417,00
3º	Esfiha de Calabresa	Esfiha	738	R$ 5.066,33
4º	Pizza Calabresa	Pizza	424	R$ 21.030,36
5º	Pizza Mussarela	Pizza	358	R$ 16.322,04

Evolução das vendas

- **Linha verde:** faturamento (escala em R$ à esquerda).
- **Barras azuis:** quantidade de pedidos (escala à direita).
- Passe o mouse sobre um ponto para ver os valores exatos daquele dia.
- No filtro **Agrupar gráfico por**, escolha **Dia**, **Semana** ou **Mês**. Para períodos longos, *Semana* ou *Mês* deixa a tendência mais clara.
- ### Situação dos pedidos

Gráfico de rosca com a quantidade de pedidos em cada etapa: **Recebido, Em preparação, Pronto, Saiu para entrega, Retirado, Finalizado e Cancelado**. Passe o mouse para ver a quantidade e a porcentagem.

Com o período terminando hoje, as fatias *Recebido*, *Em preparação* e *Saiu para entrega* mostram os pedidos **em andamento neste momento**.

Vendas por hora do dia

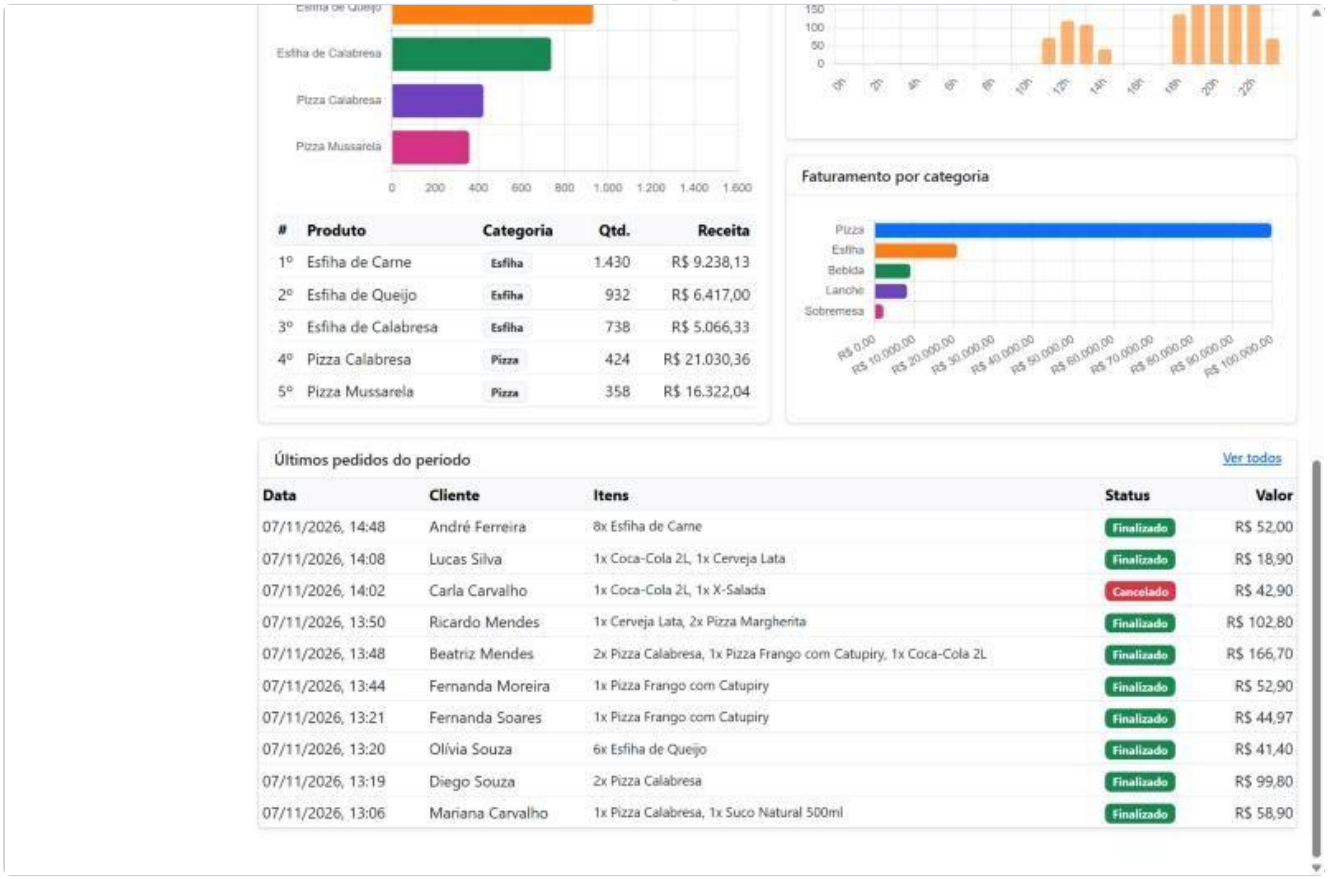
Barras com a quantidade de pedidos em cada hora (0h a 23h). Mostra os dois movimentos da pizzaria: um **almoço** menor (11h–14h) e o **jantar**, bem mais forte (18h–23h).

Faturamento por categoria

Quanto cada categoria (Pizza, Esfiha, Bebida, Lanche, Sobremesa) faturou no período, da maior para a menor.

---

6. Produtos mais vendidos e últimos pedidos



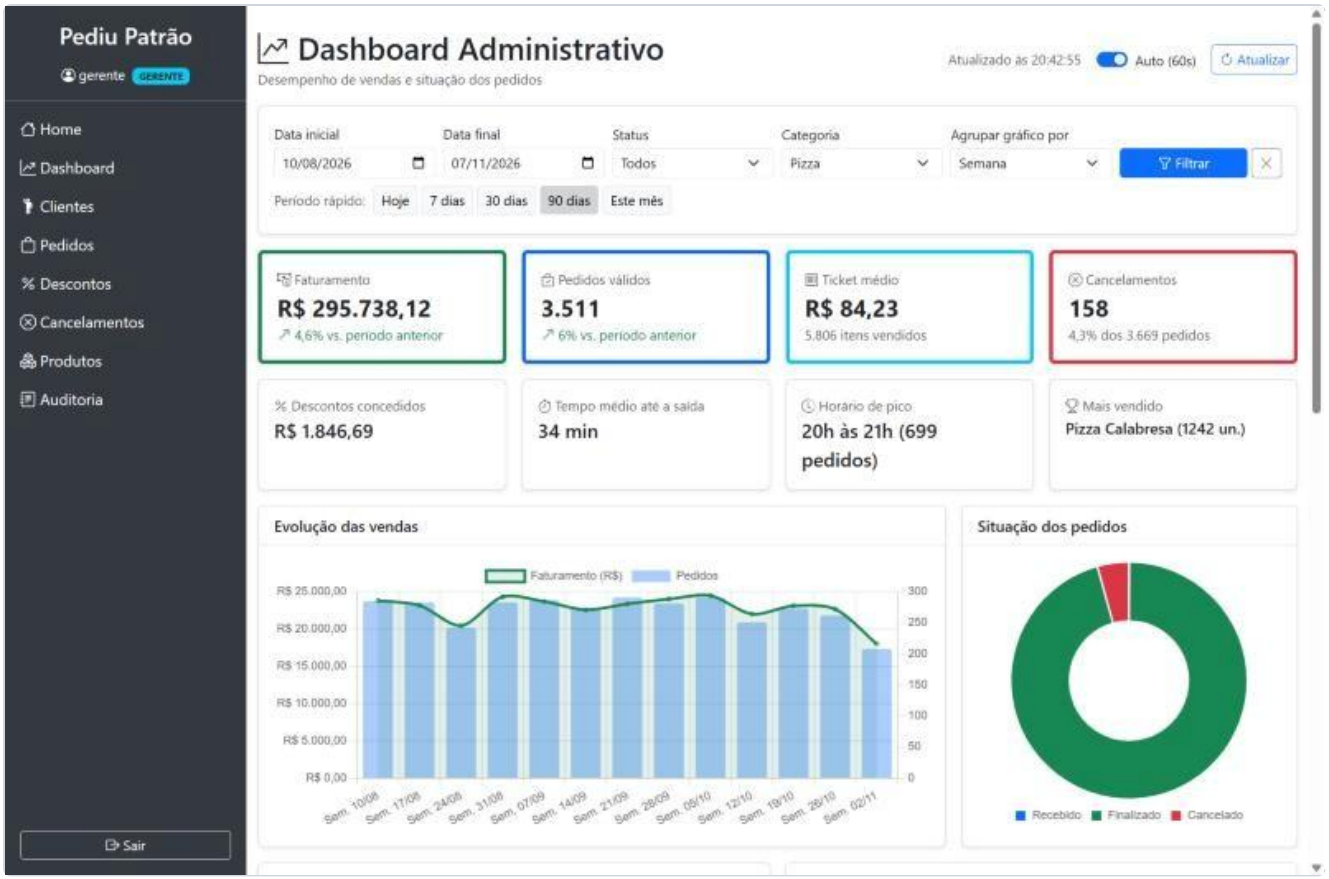
Produtos mais vendidos

O ranking mostra os **5 produtos com mais unidades vendidas**: em gráfico de barras e numa tabela com posição, produto, categoria, quantidade e receita.

---

7. Filtros

Os filtros mudam **todos** os números, gráficos e tabelas da tela de uma vez.



Como filtrar

1. Escolha a **Data inicial** e a **Data final**, ou clique num **Período rápido**.
2. Se quiser, escolha um **Status** (ex.: só *Cancelado*) e/ou uma **Categoria** (ex.: só *Pizza*).
3. Escolha como **Agrupar o gráfico** (Dia, Semana ou Mês).
4. Clique em **Filtrar**.

Controle	Uso
Período rápido	*Hoje*, *7 dias*, *30 dias*, *90 dias* ou *Este mês*. Aplica na hora. Em *90 dias*, o gráfico muda sozinho para *Semana*
Status	Mostra só os pedidos naquela etapa

Controle	Uso
Categoria	Considera só os produtos daquela categoria. O faturamento passa a somar **apenas os itens da categoria**
✕ **(Limpar)**	Volta para os últimos 30 dias, sem status nem categoria

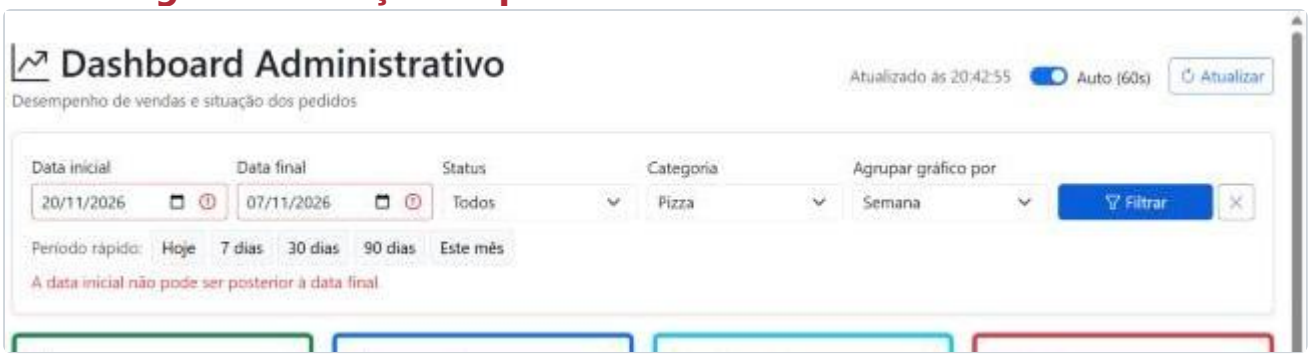
8. Atualização dos dados

Recurso	Como funciona
Auto (60s)	Ligado por padrão: os números são atualizados sozinhos a cada minuto. Desligue se quiser "congelar" a tela para analisar
Atualizar	Busca os dados na hora
Atualizado às hh:mm:ss	Mostra quando os números foram buscados pela última vez

A atualização automática pausa quando a aba do navegador não está visível, e volta ao reabrir.

---

9. Mensagens e situações especiais



Situação	O que aparece	O que fazer
Data inicial depois da final	Campos em vermelho e "*A data inicial não pode ser posterior à data final.*"	Corrija as datas
Período maior que 366 dias	"*O período máximo é de 366 dias.*"	Diminua o período
Data em branco	"*Informe a data inicial e a data final.*"	Preencha as datas
Nenhum pedido no filtro	"*Nenhum dado para os filtros selecionados.*" nos gráficos e tabelas	Amplie o período ou retire filtros
Sem conexão com o servidor	Faixa vermelha "*Não foi possível conectar ao servidor.*" e botão **Tentar novamente**	Verifique a internet/servidor e clique em Tentar novamente
Sessão expirada	Volta para a tela de login	Entre de novo

Situação	O que aparece	O que fazer
Usuário sem permissão	"*Você não tem permissão para ver o dashboard.*"	Use um usuário Gerente ou Administrador
Período anterior sem	"*sem base no período anterior*" no lugar da seta vendas	Normal no começo da operação
"Tempo médio até a saída" *Sem dados*	Nenhum pedido do período chegou à saída com	Normal se o período tem só pedidos em andamento ou cancelados

---

11. Dashboard no Grafana

Os mesmos indicadores também estão disponíveis no **Grafana**, uma ferramenta profissional de painéis. Ele é útil para deixar numa TV da loja ou para análises maiores.



Como acessar

1. Com o Docker aberto, inicie o Grafana (veja `docs/DASHBOARD.md`, seção 5.2).
2. Acesse `http://localhost:3000` e entre com o usuário do Grafana.
3. O painel **"Pediu Patrão — Dashboard Administrativo"** abre como página inicial.

Como usar

Controle	Onde	Para quê
Período	Canto superior direito (ex.: *Last 6 months*)	Mesmo papel das datas na tela do sistema
Status / Categoria / Agrupar por	Topo, à esquerda	Mesmos filtros da tela do sistema
Refresh 1m	Canto superior direito	Atualização automática (30 s, 1 min, 5 min...)
Modo TV	Acrescente `&kiosk` ao endereço	Esconde menus, ideal para monitor na parede

Como executar o projeto

1 Executar a aplicação

Pré-requisitos: Java 21 e acesso ao MongoDB. O Docker é opcional (usado no Grafana e nos testes de integração).

```
./mvnw spring-boot:run
```

Acesse `http://localhost:8080`, entre com um usuário **Gerente** ou **Administrador** e clique em **Dashboard** no menu.

2 Variáveis de ambiente

Variável	Para quê
`SPRING_DATA_MONGODB_URI` / `SPRING_DATA_MONGODB_DA`	Conexão com o banco (substituem o `application.properties`) `TABASE`
`DASHBOARD_API_KEY`	Chave que o Grafana usa para ler a API (a mesma do arquivo `grafana/.env`)

3 Grafana

```
cd grafana
docker compose up -d
```

Abra `http://localhost:3000`. O painel "Pediu Patrão — Dashboard Administrativo" já vem configurado (fonte de dados, chave da API e 17 painéis).

4 Testes automatizados

```
./mvnw clean test
```

Os testes de integração criam um MongoDB descartável em Docker; sem o Docker aberto, eles são ignorados.